

ML in Electronics Assignment

Report on

**AI-Powered Library Silence Monitor: A Multimodal
Approach to Active Speech Detection**

By:

Jaival Talati

2020HD231650P

Priyank Khandelwal

2025HD12

In Partial Fulfillment of

G513 ML in Electronics

at



**BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE,
PILANI (Rajasthan)**

(November, 2025)

Table of Contents

1. Introduction

- 1.1 Problem Statement
- 1.2 Proposed Solution
- 1.3 System Architecture Overview

2. The Visual Subsystem

- 2.1 Evolution of Strategy
- 2.2 Automated Dataset Generation
- 2.3 Visual Model Architecture
- 2.4 Temporal Smoothing

3. The Audio Subsystem

- 3.1 Data Acquisition and Feature Engineering
- 3.2 Model Architecture Strategy
- 3.3 Training Methodology

4. Multimodal Fusion & Deployment

- 4.1 Fusion Logic
- 4.2 Multi-Person Tracking
- 4.3 Synchronization (Hold Timer)

5. Performance & Results

- 5.1 Evaluation Metrics
- 5.2 Achieved Training Performance
- 5.3 Real-World Deployment Analysis

6. Conclusion & Future Scope

- 6.1 Conclusion
- 6.2 Future Scope

1. Introduction

1.1 Problem Statement

Maintaining silence in a library may seem straightforward, but in practice it is surprisingly challenging. Traditionally, the responsibility falls on library staff, who must manually walk around, identify noisy groups, and intervene when necessary. This manual monitoring approach comes with several limitations:

- **Inefficiency:** Staff cannot be everywhere simultaneously. A group might quiet down when they spot a librarian approaching, only to resume talking once the staff member leaves.
- **Subjectivity and Awkwardness:** Deciding when to intervene is often subjective and can occasionally lead to uncomfortable interactions or friction between staff and patrons.
- **Wasted Human Effort:** Instead of assisting visitors, managing resources, or performing administrative duties, staff end up spending valuable time acting as “noise police.”

Simple automated approaches, such as standalone decibel meters, offer little improvement. These systems are essentially *blind*—they only measure loudness and cannot differentiate between problematic speech (e.g., loud conversations) and harmless sounds (e.g., someone dropping a book or sneezing). Their inability to understand *context* often results in frequent false alarms, making them impractical in real-world use.

1.2 The Proposed Solution

To address these challenges, we designed and developed an **AI-Powered Library Silence Monitor**—a system that goes beyond measuring sound levels and instead attempts to *understand* whether people are actually speaking. Since relying on audio alone is unreliable due to the complex nature of real-world noise (keyboards, chair squeaks, footsteps, ventilation systems), we adopted a **multimodal strategy** combining both visual and audio intelligence.

- **Visual Analysis (Eyes):** Detects faces and evaluates whether a person’s lips are moving.
- **Audio Analysis (Ears):** Classifies incoming sound as “Speech” or “Non-Speech.”

The final decision is made *only when both* modalities indicate talking, dramatically reducing false positives. For example, if the audio subsystem misinterprets a creaky chair as speech, the visual subsystem checks for lip movement and blocks the alert. This synergy ensures more accurate and context-aware monitoring.

1.3 System Architecture Overview

The system is composed of two core AI subsystems:

1. **Visual Subsystem (VSD):**

Detects faces using a camera feed, generates cropped mouth regions, and classifies them as **Talking** or **Not Talking** using a custom CNN.

2. **Audio Subsystem (VAD):**

Converts raw audio into Log-Mel spectrograms and classifies each sample using a custom **ResNet-18**, trained to distinguish **Speech** from **Non-Speech**.

Although the audio model is powerful, environmental noise can still confuse it. Therefore, the visual subsystem acts as a crucial verification layer during fusion.

2. The Visual Subsystem

2.1 Evolution of the Strategy

The first approach attempted to classify talking behavior using geometric features extracted from facial landmarks—specifically, distances between upper and lower lip points. Although simple, this method lacked robustness and frequently misclassified actions like yawning, smiling, or chewing as speech.

To overcome these limitations, we transitioned to a **deep learning–based approach**. By training a CNN directly on mouth-region images, the model learns subtle visual cues and

temporal patterns that are difficult to capture through handcrafted features. This shift significantly improved reliability.

2.2 Automated Dataset Generation

Manually labeling thousands of frames is neither scalable nor efficient. To address this, we built an **automated dataset-labeling pipeline** using MediaPipe Face Mesh.

Labeling Logic:

- Compute the **Lip Gap (LG)** between the vertical centers of the upper and lower lips.
- If **LG > 0.2**, label the frame as **Talking**.
- If **LG < 0.05**, label the frame as **Not Talking**.
- Frames falling between these thresholds are discarded to avoid ambiguity.

This automated process produced thousands of high-quality, 64×64 grayscale mouth crops suitable for training.

2.3 Visual Model Architecture

The CNN used for classification follows a compact yet effective architecture:

- **Input:** 64×64 grayscale mouth images
- **Feature Extraction:** Two blocks of Conv2D → ReLU → MaxPooling
- **Classification Head:** Flatten → Dense (ReLU) → Dense (Sigmoid)
- **Output:** Binary label (Talking / Not Talking)

This structure provides a good balance between accuracy and computational efficiency, making it suitable for real-time processing.

2.4 Temporal Smoothing

Per-frame predictions can be noisy due to lighting variations, motion blur, or brief facial movements. To stabilize predictions, we use:

- A **0.5-second rolling buffer** of recent predictions.
- A decision rule: if **>80%** of frames in the buffer indicate “Talking,” the final output is set to **Talking**.

This approach filters out jitter and improves temporal consistency.

3. The Audio Subsystem

3.1 Data Acquisition and Feature Engineering

The audio dataset consists of approximately **20 hours** of recordings—around **60,000 spectrogram samples**, evenly split between:

- **Speech:** ~30,000 samples from the LibriSpeech “clean” subset
- **Non-Speech:** ~30,000 samples of environmental noise from UrbanSound8K

Each raw audio segment is converted into a **Log-Mel Spectrogram** using:

- Sample Rate: 16 kHz
- FFT Window: 400 samples (≈ 25 ms)
- Hop Length: 160 samples (≈ 10 ms)
- Mel Bands: 64
- Output Shape: **64 × 97**

These spectrograms serve as the input images for the neural network.

3.2 Model Architecture Strategy

We implemented a custom **ResNet-18** in TensorFlow/Keras for spectrogram classification.

Key design decisions include:

- Replacing the standard 7×7 convolution stem with a 3×3 kernel to preserve high-resolution audio features.
- Using residual blocks to ensure stable gradient flow during training.
- Employing **Global Average Pooling** instead of large fully connected layers, reducing total parameters to ~11M and improving generalization.
- Training from scratch to tailor the model to domain-specific data.

3.3 Training Methodology

- Dataset carefully balanced: Speech : Non-Speech = **1 : 1**
- Optimizer: **Adam (lr = 0.001)**
- Loss Function: **Categorical Cross-Entropy**
- Regularized with:
 - **EarlyStopping**
 - **Learning Rate Scheduler**

These measures ensured strong convergence and minimized overfitting.

4. Multimodal Fusion and Deployment

4.1 Fusion Logic

The final speech-detection decision uses a **gated three-stage process**:

1. **A face must be present in the frame.**
2. **The visual subsystem must detect lip movement.**
3. **The audio subsystem must classify the sound as speech.**

Only when *all three* conditions are satisfied is a person labeled as **Speaking**.

This greatly reduces false alarms caused by:

- Visual noise (e.g., chewing, yawning)
- Audio noise (e.g., chair squeaks, typing, footsteps)

4.2 Multi-Person Tracking

The system employs **Centroid Tracking** to handle up to **five people** simultaneously. Each person receives:

- A unique ID
- Independent visual-state buffers
- Individual predictions

This ensures reliable, trackable monitoring even in busy scenes.

4.3 Synchronization (Hold Timer)

Since audio operates on **0.96-second** chunks and video runs at **30 FPS**, we use a **1.5-second hold timer**. Whenever the audio system detects speech, this timer keeps the “Speech Active” state high long enough for the visual subsystem to interpret the corresponding frames.

5. Performance and Results

5.1 Evaluation Metrics

All models were evaluated on a held-out test set using:

- Test Accuracy
- Test Loss

5.2 Achieved Performance

The custom ResNet-18 achieved:

- **Test Accuracy:** 99.50%
- **Test Loss:** 0.0433

These results indicate strong discriminative capability on clean evaluation data.

5.3 Real-World Deployment Observations

Real-world environments introduce challenges not present in curated datasets:

- LibriSpeech is very clean compared to typical libraries.
- Real spaces include echoes, mixed background noises, and overlapping sources.
- Mechanical noises (chair movements, typing, sliding books) can mimic speech patterns in spectrograms.

The multimodal fusion approach effectively mitigates these issues by requiring visual confirmation before declaring speech.

6. Conclusion and Future Scope

6.1 Conclusion

The AI-Powered Library Silence Monitor demonstrates that **multimodal intelligence vastly outperforms single-sensor systems** for noise detection in quiet, human-centric environments. By combining a lip-movement classifier with a custom audio-based ResNet-18, the system achieves highly reliable, context-aware speech detection with minimal false positives. This enables libraries to maintain silence more efficiently and unobtrusively.

6.2 Future Scope

1. Beamforming:

Integrating a microphone array would make it possible to localize sound sources and associate detected speech with specific individuals.

2. **Edge Deployment:**

The lightweight architectures are well-suited for devices like the NVIDIA Jetson Nano or Raspberry Pi, enabling offline, privacy-preserving deployment without relying on cloud resources.